

```
$ ./percona-live-2026
```

Your LLM Will **Drop** That Table.

Making AI-Assisted Database Work Safe

David Murphy · Senior Database Engineer, Motion

Percona Live 2026 · Wed May 27, 4:20pm · Boole

// WHOAMI

Who is David Murphy

Senior Database Engineer at Motion

deep production experience across Mongo, Postgres, MySQL, Elasticsearch, and Oracle

Former Practice Manager for MongoDB at Percona

and Lead DBA at ObjectRocket, the high-performance Mongo-as-a-Service offering

Earlier: NoSQL / MySQL Architect at Electronic Arts

(yes, I probably worked on that game you're thinking about)

20+ years • hundreds of talks, webinars, and blogs

Mongo Master Alumni — one of the original Mongo Masters

Focused on DBRE and database security

the operational reality of running databases people depend on

// where i work

What is Motion?



If we haven't met yet, we're Motion.

A SaaS analytics platform that ingests billions of ad performance records from Meta, TikTok, YouTube, and LinkedIn — and an AI agent, Runneth, that sits on top of it to tell marketing teams in plain English which of their ads are actually working and what to ship next.

\$14B

in media spend analyzed every year
by Motion's customers

Names you know

HEXCLAD

RIDGE

VUORI

ClickUp

AG1®

MARY RUTH'S®

70+

employees, remote-first, distributed
across the globe

What This Talk Is

Your developers already have an AI assistant connected to production. Today, this week, right now. It will:

- Run an aggregation that scans a collection with **500 million documents**
- Suggest a **join with no index coverage** on either side
- Write a **migration that destroys data if it runs out of order**

This is not hypothetical. It happens. It will happen to your team.

This talk is the workflow I use to stop it — **database-agnostic**, at the IDE, the PR, and the deploy.

Policy And Review Aren't Enough

Two instincts we reach for — and why both fail.

“Write a policy.”

- The AI has no context about your schema, indexes, or what a query costs at scale.
- A policy doc loses to a developer with a deadline. Every time.

“Review it more carefully.”

- A diff doesn't show that **events** has 300M rows.
- “Looks fine” is the most dangerous comment in any PR.

The fix: **same rule, enforced earlier and at every stage.**

The Categories That Cause Incidents

1. **Unbounded scans** — no limit, no index hint, no shard key in the filter
2. **Joins without index coverage** — \$lookup / JOIN against an unindexed foreign side
3. **Write amplification** — updateMany / UPDATE where the filter is wrong
4. **Destructive migrations** — irreversible drops, column rewrites, \$unset runs
5. **Out-of-order migrations** — B assumes A ran; CI ran them backwards
6. **Hallucinated schema** — code written against fields that don't exist
7. **Silent index swaps** — replacing a compound index with single-fields because "cleaner"

The model will produce all seven without remorse.

The Story: One Developer, One Feature

A developer picks up a ticket Tuesday:

"Show users their activity for the last 30 days."

The activity collection: ~300M documents.

One good compound index: (user_id, ts).

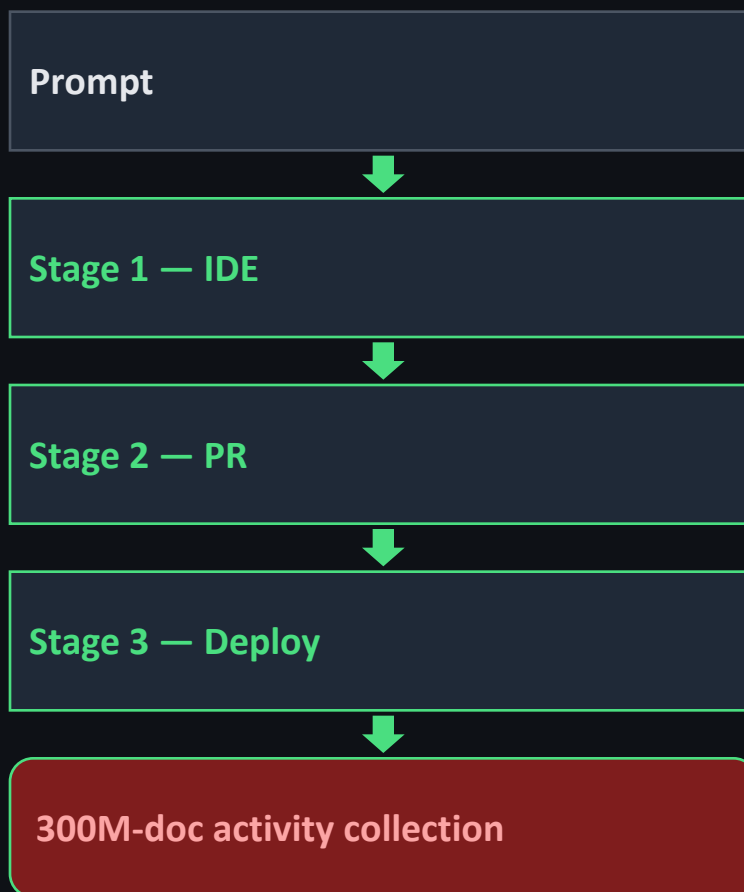
A half-finished migration from last sprint.

They open the IDE. AI assistant connected.

They type one prompt.

What happens next — twice.

Once without safety. Once with.



Stage 1 — IDE — Without Safety

The AI:

1. Opens a DB connection with `.env` credentials — usually the dev's personal user, **often with write access**.
2. Hallucinates the field is `userId`. It's actually `user_id`.
3. Writes an aggregation: `$group` on country first, no `$match` on `ts`. **COLLSCAN on 300M docs**.
4. Local cluster grinds for **90 seconds**.
5. "Optimizes" by suggesting `dropIndex('user_id_1_ts_1')`.
6. Drafts a single-direction migration. **No rollback**.

By the time the PR opens, the design is already broken.

// THE ONE RULE

The database connection your AI uses must be read-only by default.

Not "in production." Always. Until you deliberately escalate.

Postgres

```
CREATE ROLE ai_local LOGIN
  PASSWORD : 'pw';
GRANT pg_read_all_data TO ai_local;
ALTER ROLE ai_local
  SET statement_timeout = '5s';
```

MongoDB

```
db.createUser({
  user: "ai_local",
  pwd: passwordPrompt(),
  roles: [{ role: "read",
             db: "app" }]
})
```

// STAGE 1 — IDE — WITH SAFETY

Five files. Twenty minutes. Seven failures gone.

File	What it does
DB user (ai_local)	Permission floor — read-only, statement_timeout
.mcp.json	MCP connects with --readOnly, version pinned
CLAUDE.md	Rules every turn: EXPLAIN first, propose-don't-create indexes
.claude/skills/*.md	Playbooks: schema-pin, explain-first, index-change-guard
.claude/settings.json + hooks	Block index changes, write-shaped aggs, COLLSCANs

// THE LOCAL FILES (database-agnostic)

Configure once. Checked in. Runs every turn.

.mcp.json // checked in

```
{ "mcpServers": {  
  "mongodb": { "command": "npx", "args": [  
    "-y", "mongodb-mcp-server@1.4.2",  
    "--readOnly",  
    "--connectionString",  
    "${DB_URI_AI_LOCAL}" ] } } }
```

CLAUDE.md // checked in

- Before any aggregation/JOIN on activity, users, sessions: run EXPLAIN and confirm docs-examined / rows-returned ≤ 100 .
- Propose index changes as migration files. Never call createIndex / dropIndex directly.
- Verify fields against .schemas/<table>.json.

.claude/hooks/db-guard.sh

```
INPUT="$(cat)"  
jq -e '.tool_name |  
test("(create|drop)Index")' \<<<"$INPUT" >/dev/null && \  
{ echo '{"decision":"block"}'; exit 0; }  
jq -e '.tool_input|..|objects|select(  
has("$out") or has("$merge"))' ...
```

Same pattern in Postgres: read-only role, statement_timeout, an MCP that wraps every query in **BEGIN READ ONLY; EXPLAIN; ROLLBACK**, plus an AST hook that blocks DROP / TRUNCATE / unbounded UPDATE.

// STAGE 1 – IDE – REPLAY WITH SAFETY

Same prompt. Same AI. Different outcome.

- 1 AI opens DB connection → **ai_local**, read-only. **dropIndex** is not a callable tool.
- 2 Schema lookup → **schema-pin** skill points to **.schemas/activity.json**. AI uses **user_id**.
- 3 First aggregation has no \$match on ts → **hook blocks**. AI must add the filter.
- 4 Rewrites with \$match on (user_id, ts). EXPLAIN: **IXSCAN**, docs-examined 18k, ratio 6. ✓
- 5 Proposes an index? Emits a **migration file** with up, down, EXPLAIN diff. No direct call.
- 6 PR opens with code + tests + migration. **The 300M-doc collection was never touched.**

Same rule that fired: **no query reaches the DB without proving its cost first.**

// STAGE 2 – PR – WITHOUT SAFETY

The PR shows what looks plausible.

THE PR SHOWS

- An aggregation that looks plausible
- A migration: ALTER TABLE activity ADD COLUMN...
- One reviewer, half-attention, 4:50pm Thursday

WHAT THEY MISS

- Migration has no rollback
- Depends on a column in an unmerged migration
- \$lookup against users with no index on foreign side
- New compound index replaces one used by 3 other queries

“LGTM.”

// STAGE 2 – PR – WITH SAFETY

Same rule, different gate.

The same safety rule runs in CI on every PR touching SQL / aggregation / migration files.

Schema-pin CI

Fails the PR if a query references a field not in .schemas/

EXPLAIN bot

Runs every new aggregation on shadow data. Auto-fails on docs-examined / rows-returned > 1000

Migration linter

No down → fail. Depends on unmerged migration → fail. Index replacement without justification → fail

Required reviewer

Auto-assigned for any migration touching the top-10 large tables

Reviewers see a risk score and an EXPLAIN diff — not raw code.

// STAGE 3 — DEPLOY — WITHOUT SAFETY

CI runs migrations alphabetically.

Migrations are not alphabetical — they have dependencies. CI doesn't know.

- | | | |
|---|--|-------------------------------------|
| 1 | <code>2026_05_27_add_archived.sql</code> | runs FIRST (alphabetical) |
| 2 | <code>2026_05_20_create_archive_log.sql</code> | force-pushed late, runs SECOND |
| 3 | <code>add_archived</code> reads <code>archive_log</code> | table doesn't exist yet — half-runs |
| 4 | Partial state: half rows <code>archived=true</code> | cleanup job runs on wrong set |

DATA LOST.

Or the \$unset ran. The old field is gone. Backup is from 6 hours ago.

// STAGE 3 – DEPLOY – WITH SAFETY

Same rule, third gate.

- ✓ **Dependency-ordered migrations**
Every migration declares `depends_on::`. Refuse B if A hasn't run.
- ✓ **Dry-run on staging first**
EXPLAIN against snapshot. Fail if docs-examined exceeds budget.
- ✓ **Backup checkpoint**
Irreversible ops (drops, `$unset`, type changes) snapshot BEFORE.
- ✓ **Feature flag ramp**
New write paths ramp on one shard / one tenant first.
- ✓ **Rollback required**
down script runs on staging in CI BEFORE up is allowed in prod.

If a destructive op gets here, it still costs a human one deliberate click.

// THE PRINCIPLE

Same Rule. Three Stages.

No query reaches the database without proving its cost first.

Stage	Where the rule lives	What it does
IDE	DB role + MCP + CLAUDE.md + hooks	Stops the bad query from being written
PR	Schema-pin CI + EXPLAIN bot + linter	Stops the bad query from being merged
Deploy	Dep-ordered migrations + dry-run + backup	Stops the bad query from being run

Write it once. Three stages share it. No duplication, no drift.

// AVOID

Anti-Patterns

1

--yes / auto-approve on tool calls.

Single biggest cause of AI-DB incidents in 2025. Off, always.

2

npx -y <mcp-server> with no version pin.

Silent supply-chain rollover.

3

Connecting the AI with your admin user

"because it's just my laptop."

4

Skipping the hook when it false-positives.

Fix the matcher; don't disable the gate.

5

Trusting the AI's EXPLAIN summary.

Re-read the JSON yourself.

6

The system-prompt-is-safety fallacy.

"We told it to be careful." Guidance $\neq$ enforcement.

// THE TALK IN TWO LINES

Read-only DB user.

Same EXPLAIN rule at IDE, PR, and deploy.

// HONEST CAVEATS

- Migration files written directly in the editor still bypass the MCP tool.
- Queries that pass EXPLAIN can still degrade under production cardinality.
- Schema changes above your size threshold still need a DBA review.

One rule. Three stages. Written once, enforced everywhere.

Twenty minutes of setup per repo. The AI stays genuinely useful all day. And it will not drop your table by accident.

// one more thing

Motion is hiring

Motion is looking for rockstars.

If anything in this talk made you nod, we should talk.

- ›› Senior engineers — backend, full-stack, infra
- ›› DBRE & platform engineering — yes, my team
- ›› Data, ML, growth, design, GTM
- ›› Remote-first. Globally distributed.

Grab me after the talk, or email david.murphy@motionapp.com

// scan to apply

 Motion



Visit our [careers page](#) to learn more



Explore our [new feature releases](#)

Careers

motionapp.com/careers

// END

Thank you.

David Murphy

Senior Database Engineer, Motion

email david.murphy@motionapp.com

twitter @dmurphy_data

github dbmurphy

REPO — Five local files, CI gates, deploy checks

github.com/dmurphy/percona-live-2026-ai-db-safety

Includes MongoDB + Postgres variants.

Questions???